Project: Wedding Invitation and Gift Management System (WIGMS)

Second Progressive Report

V1.1

GROUP 3		
NAME	**MATRICULE**	**PERCENTAGE PARTICIPATION**
TCHOUNGA NGATCHOU MARIE CLAIRE	26S022	25%
MASSADO DJOKO AICHA CLAUDIA	26S033	25%
NKOMPNANG GUILLAUME JUNIOR	26S017	25%
AVOUTIA DONGMO BRAYANN	26S028	25%

SUPERVISOR: MR MBIETIEU AMOS

Academic Year: 2025/2026

Table of Contents

1. Introduction…….3

2. Gantt Chart ……..3

3. System Design……3

4. Sequence diagram……4

5. Deployment diagram……4

6. Class diagram……6

7. Work distribution……6

8. User Interface……..6

9. Module One……..7

10. GitHub Repository……10

11. Module Two……..13

12. Repository Link……..14

13. Graphic User Interface……15

14. Tools And Technologies used……..15

15. Difficulties encountered and Proposed Solutions……...16

16. Next steps of the project……...17

17. Conclusion……..17

1. Introduction

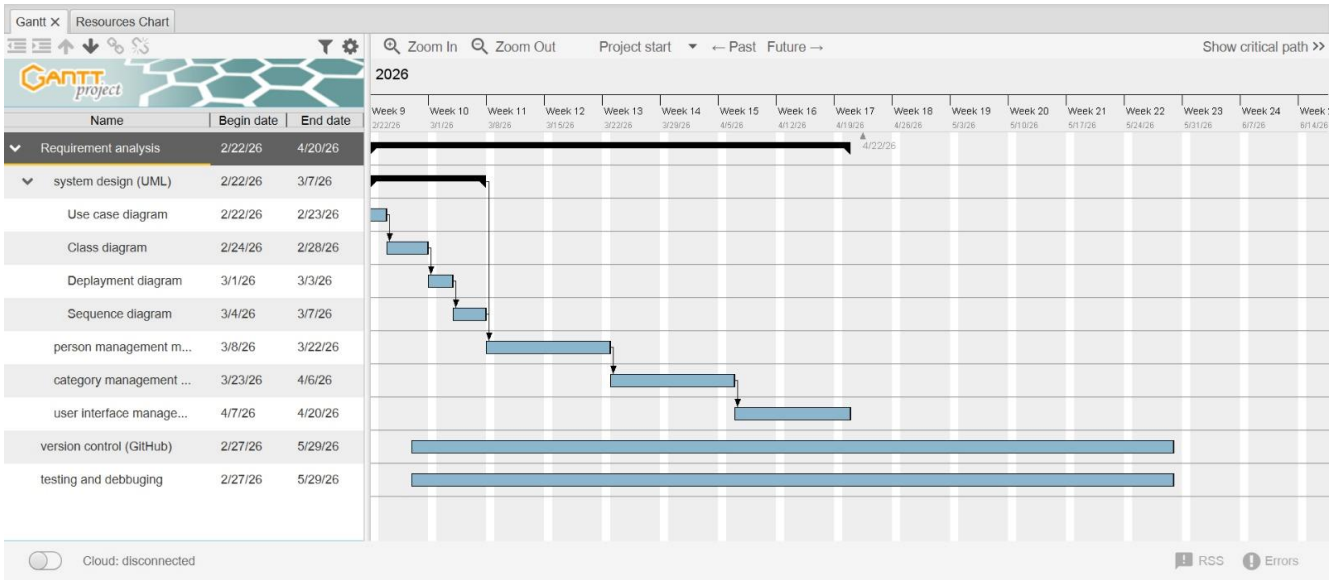
The aim of this project is to design and implement a Wedding Invitation and Gift Management System using the C programming language. The system is developed using a modular approach, allowing each component to be built, tested, and improved independently.

This second report presents:

- The corrected version of Module 1: Person Management based on supervisor remarks
- The implementation of Module 2: Category Management

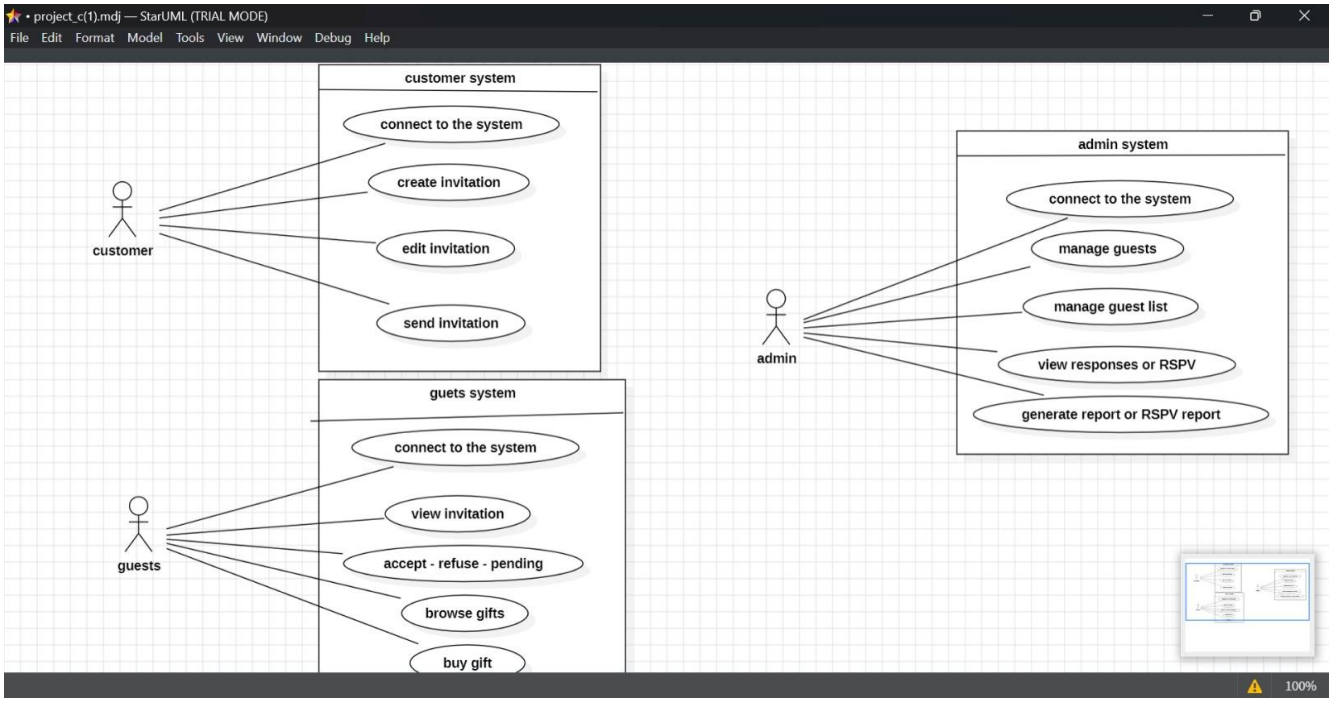
Special attention was given to improving the user interface, ensuring better usability before proceeding with advanced modules.

2. Gantt Chart



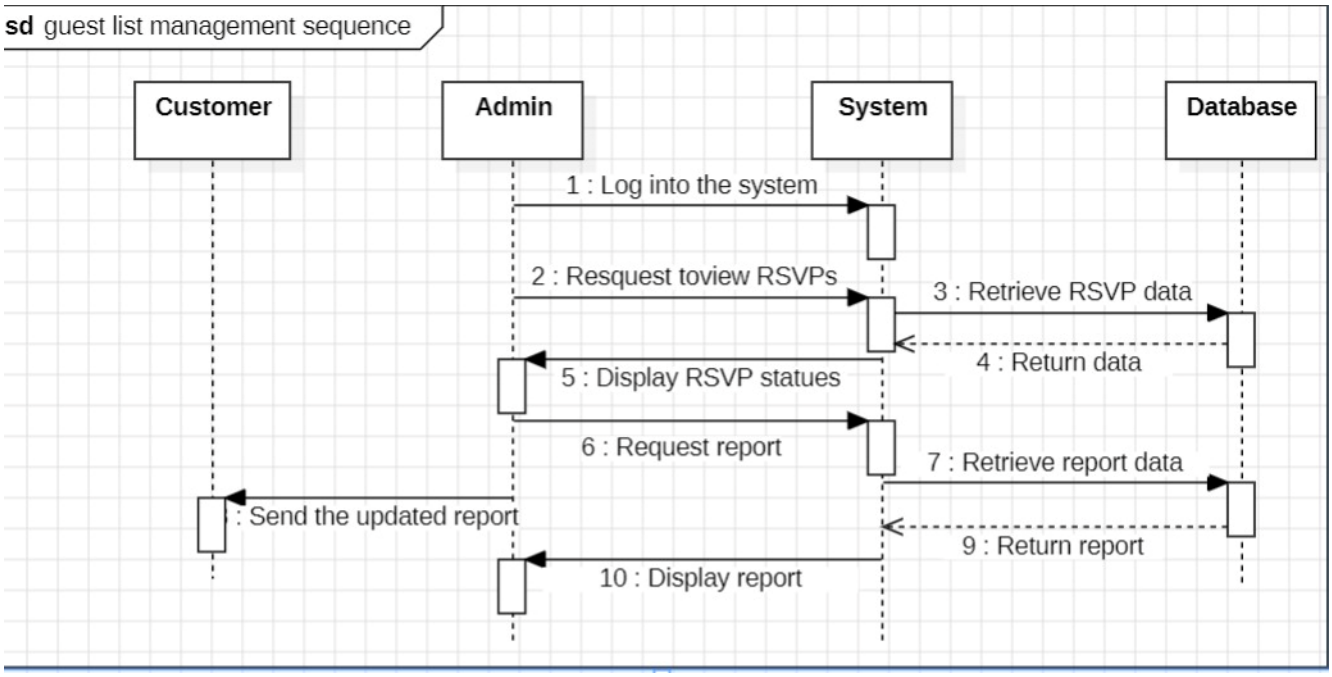
3. System Design

3.1 Use Case Diagram

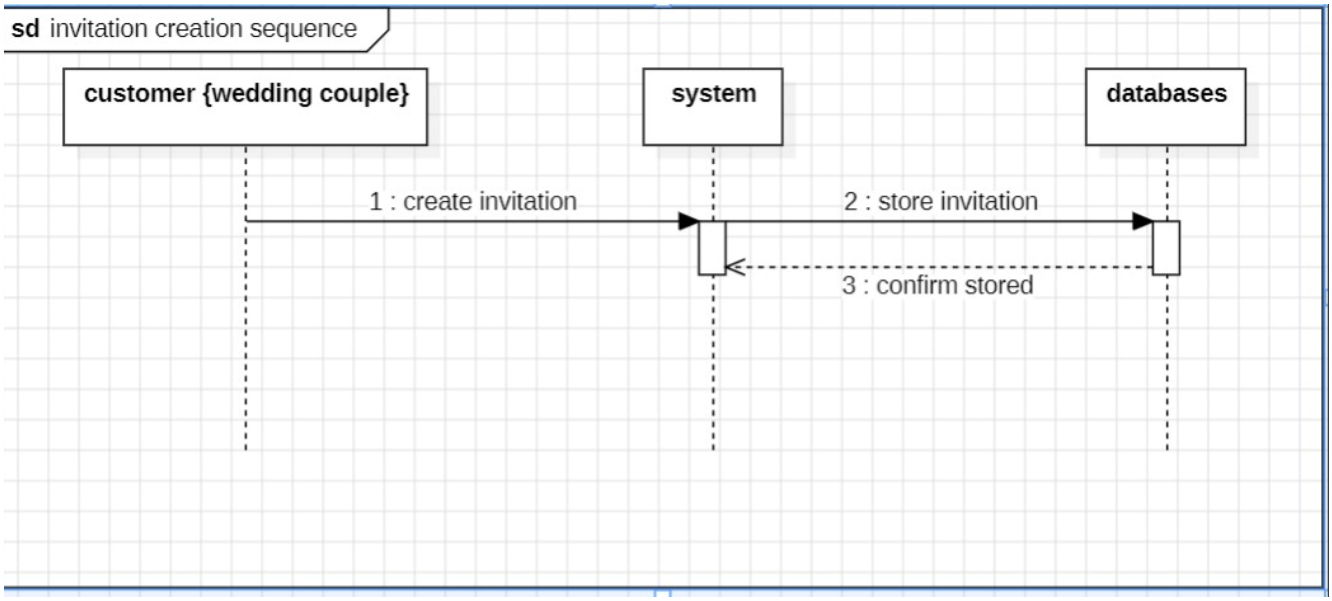


3.2 Sequence Diagram

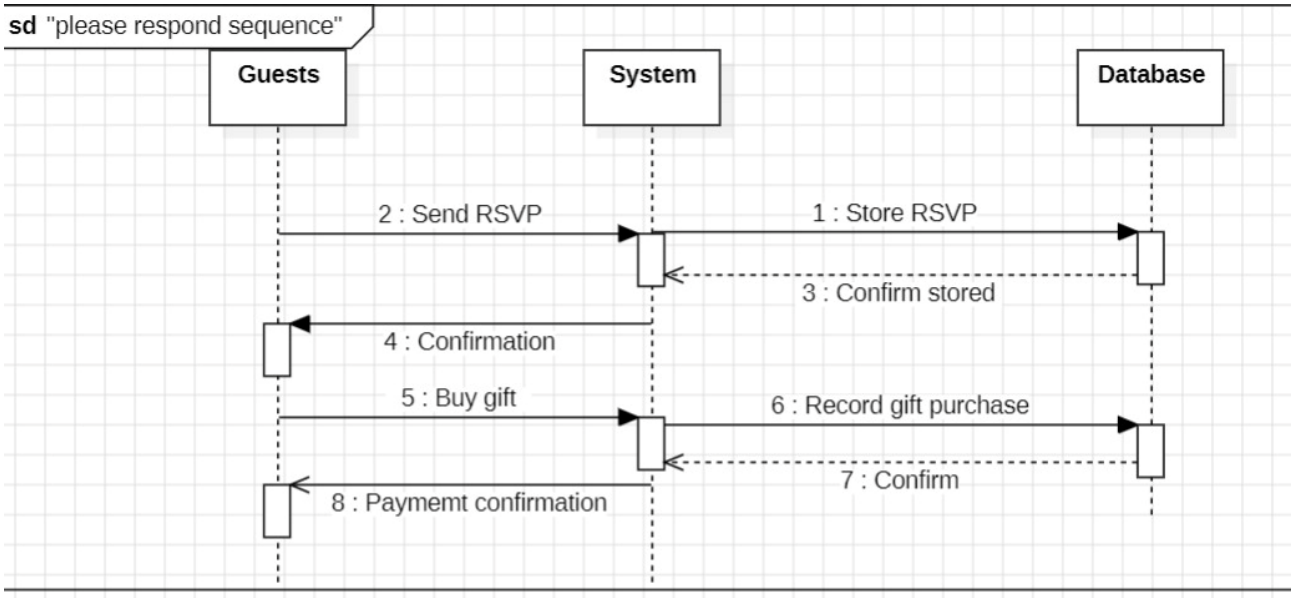
3.2.1 Guest list management sequence



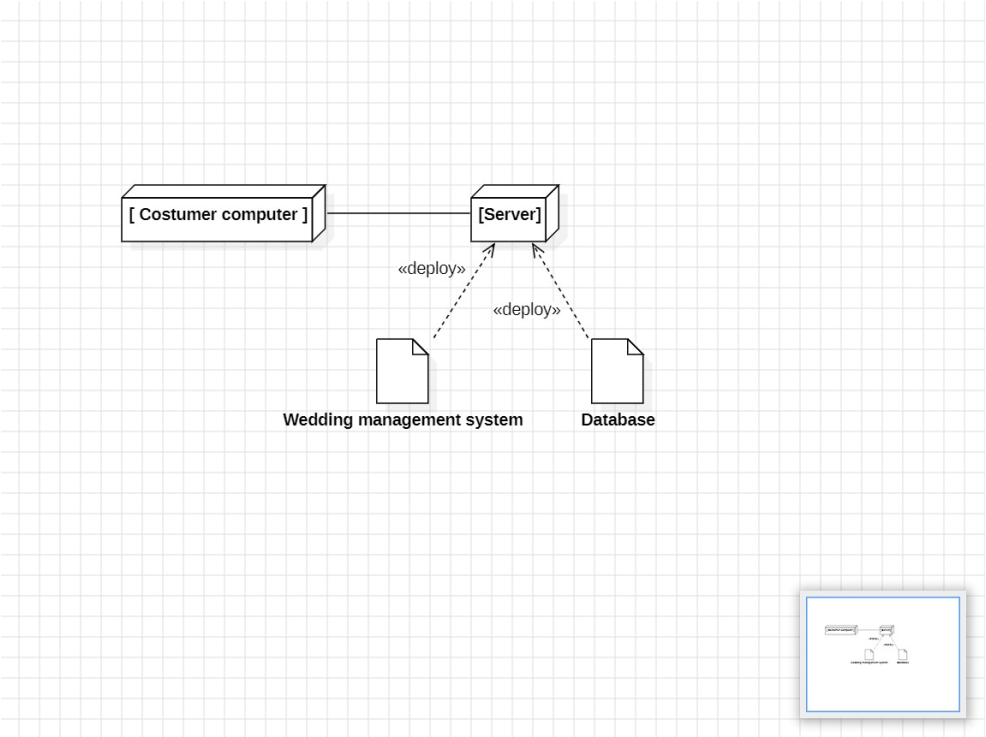
3.2.2 Invitation creation sequence



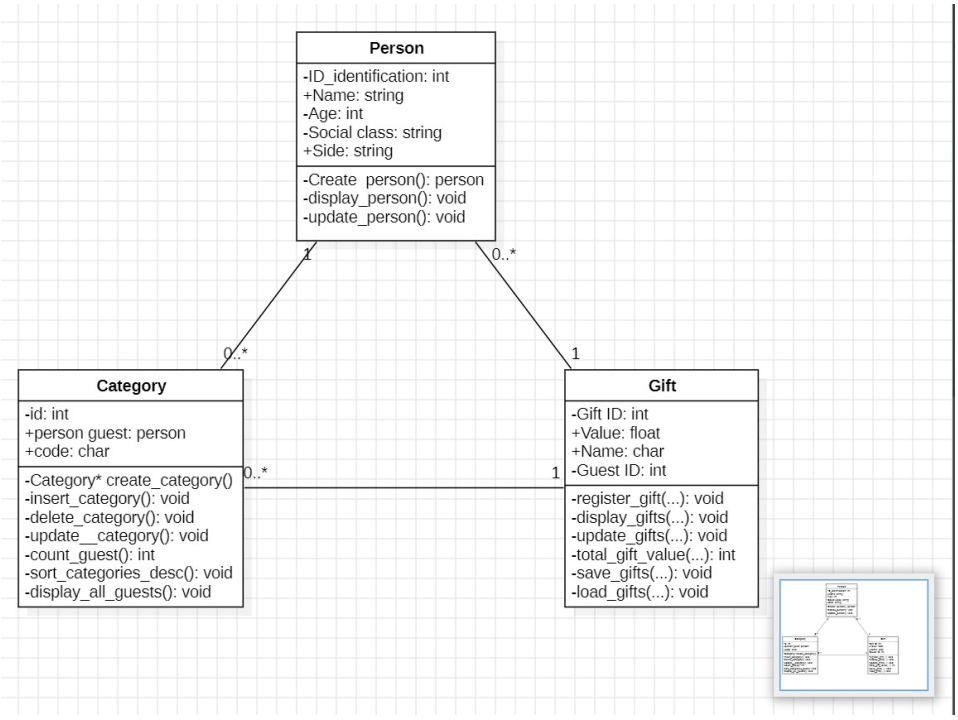
3.2.3 Respond sequence



4. Deployment Diagram



5. Class Diagram



6. Work Distribution

Group Member	Role	Assigned Tasks
Tchounga Ngatchou Marie Claire	Module Design	Designed Category structure& User Interface
Avoutia Dongmo Brayann	Function Implementation	Implemented add_category()
Nkompnang Guillaume Junior	Display Function	Implemented display_categories()
Massado Djocko Aicha	Update Function	Implemented update_categories()

7. User Interface

To improve interaction with the system, a simple user interface was introduced using standard input/output functions in C. The interface allows users to navigate through the program using a menu-driven approach.

```c
#ifndef TYPES_H
#define TYPES_H

typedef enum{
    LE,
    LA
}side;

typedef struct{
    int id;
    char name[100];
    int age;
    char social_class[50];
    side side;
}Person;

typedef struct category{
    int id;
    char code[50];
    Person guest[4];
    struct category *next;
}Category;

typedef struct{
    int gift_id;
    char name[100];
    float value;
    int guest_id;
}Gift;

#endif
```

MODULE 1 : PERSON MANAGEMENT

1. Objectives of the Module

- Define a Person structure to store guest information.
- Implement a function to create a new person.
- Implement a function to display a person's information.
- Implement a function to update a person's information.
- Apply structured programming concepts using C language.

2. Person Structure

A Person represents an invited guest. The structure stores basic information such as ID, name, age, social_class.

```c
typedef struct{
    int id;
    char name[100];
    int age;
    char social_class[50];
    side side;
}Person;
```

3. Implemented Functions

a. create_person ()

```c
Person create_person(){
    Person P;
    printf("\n- - - Guest Input - - -\n");
    printf("ENter an ID\n");
    scanf("%d",&P.id);
    getchar();

    printf("\n- - - Guest Name - - -\n");
    printf("Enter a name\n");
    fgets(P.name,sizeof(P.name),stdin);


    printf("\n- - - Guest AGE - - -\n");
    printf("ENter your Age\n");
    scanf("%d",&P.age);
    getchar();

    printf("\n- - - Guest Social Class - - -\n");
    printf("Enter your Social Class\n");
    fgets(P.social_class,sizeof(P.social_class),stdin);

    printf("\n - - -Guest side- - -\n");
    printf("Enter 0 for LE\n");
    printf("Enter 1 for LA\n");
    scanf("%d",&P.side);
    return P;
```

b. display_person ()

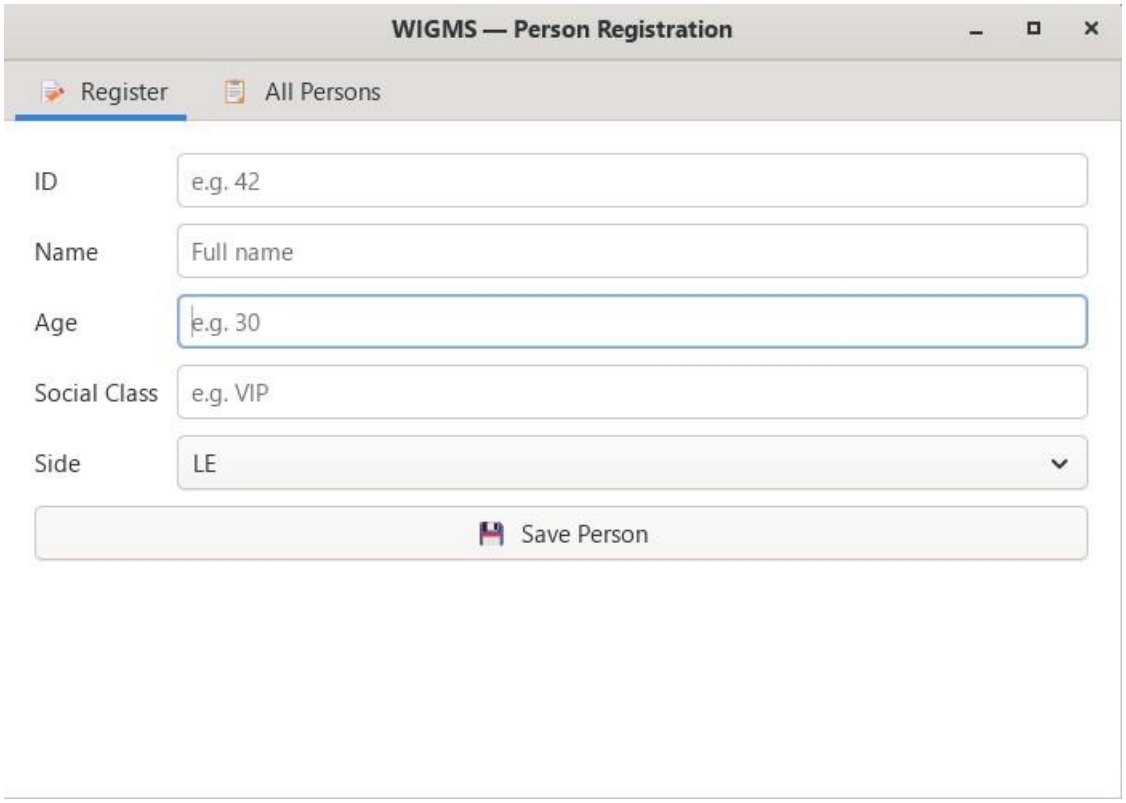
```c

void display_Person(Person P)
{
    int side_choice;
    printf("\n- - - Guest details- - -\n");
    printf("ID:%d\t | Age:%d\t |",P.id,P.age);

    if(P.side==0)
    {
        printf("side 'LE'\t");
    }
    else
    {
        printf("side 'LA'\t");
    }
    printf("| Name:%s | Social_class:%s ",P.name,P.social_class);
}

```

4. Graphic User Interface

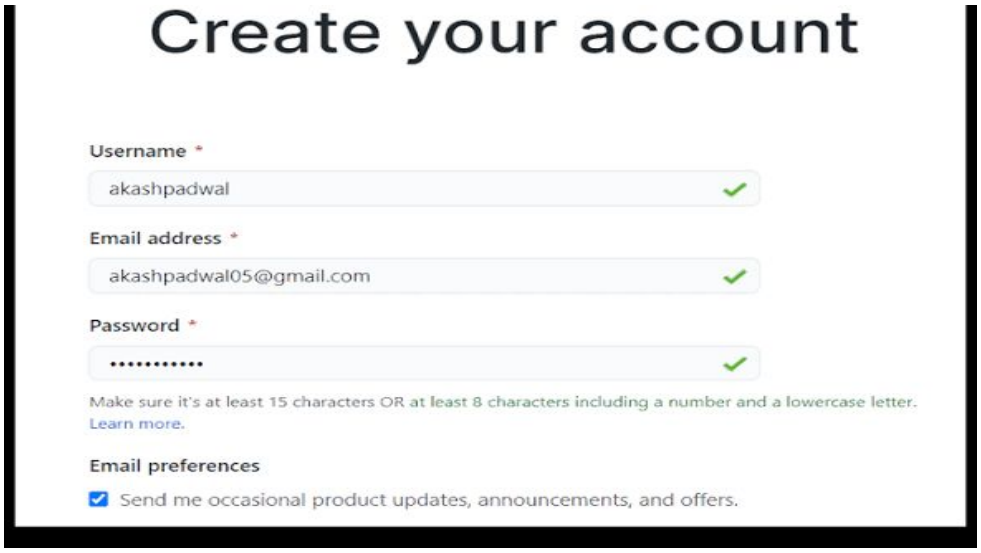


5. Setting Up a GitHub Repository

To create a GitHub repository using the web interface, follow these steps:

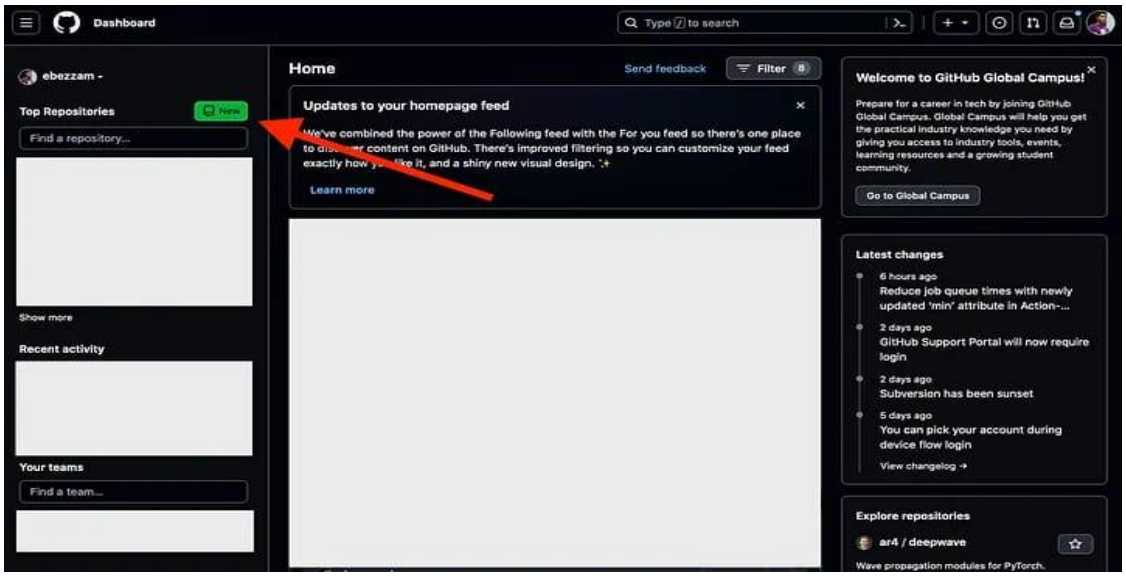
Step 1: Creating the GitHub accounts

To sign up for an account, navigate to https://github.com/ and follow the prompts. To keep your GitHub account secure you should use a strong and unique password.



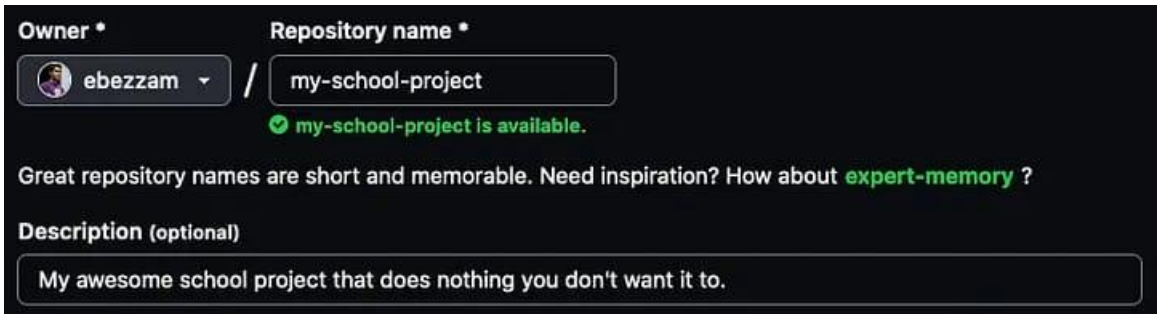
Step 2: Creating the Repository

Start a new repository in the upper-right corner of any page, select the plus (+) icon, then click New repository



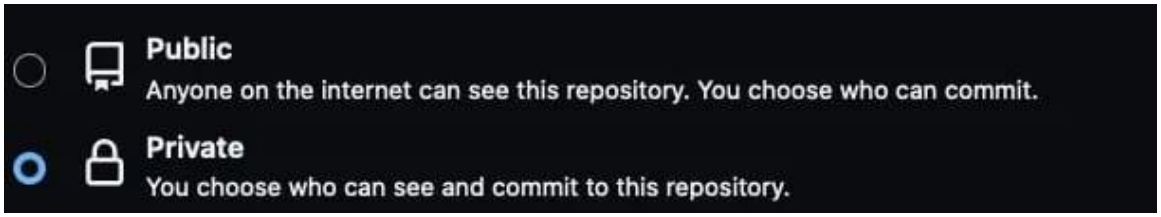
Step 3: Specify an owner

If you belong to any organizations, use the "Owner" dropdown menu to select the account (your personal account or an organization) that will own the repository. Enter a name — type a short, memorable name for your repository in the "Repository name" box.



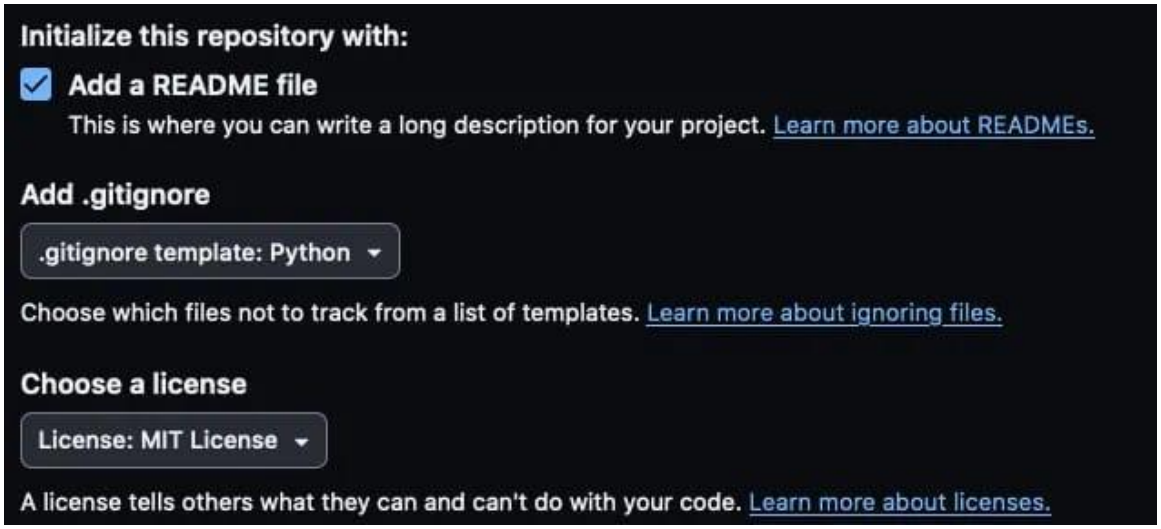
Step 4: Choose repository visibility

Decide whether the repository will be Public (anyone can see it) or Private (only you and collaborators you choose can access it).



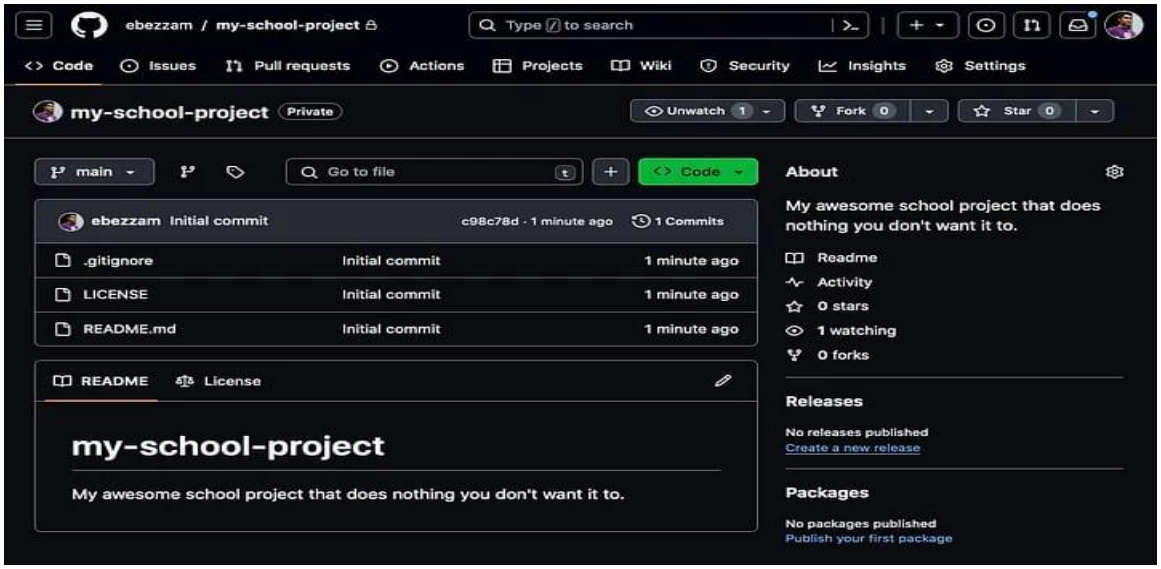
Step 5: Initialize the repository

Select Add a README file to create a document that describes your project. You can also optionally add a .gitignore file (to exclude specific file types from version control) and a license.



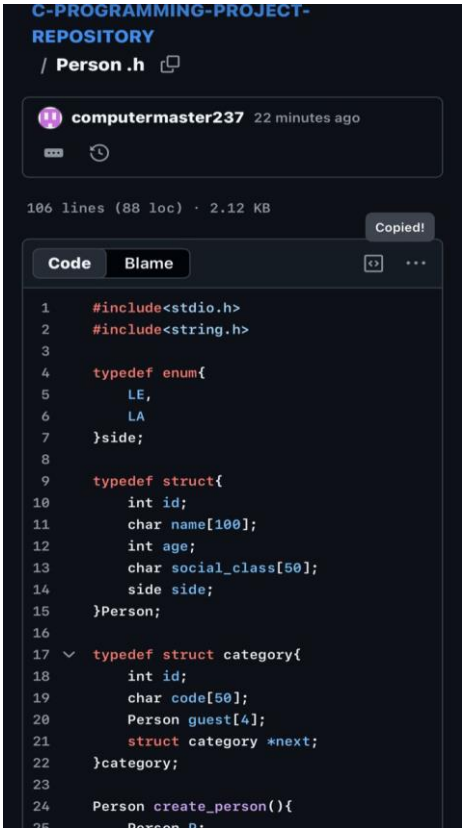
Step 6: Create the repository

Click the green Create repository button at the bottom of the page.



Step 7: Start using your repository

GitHub repository is now created and ready for you to start adding files and managing your project with version control. You can then clone the repository to your local machine using the provided URL to work with it locally.



MODULE 2: CATEGORY MANAGEMENT

1. Objectives

- Organize guests into categories
- Use linked lists for dynamic storage
- Improve system structure

2. Category Structure

```c
typedef struct category{
    int id;
    char code[50];
    Person guest[4];
    struct category *next;
}Category;
```

3. Implemented Functions

3.1 Create_Category

```c
Category* create_category() {
    Category* new_node = (Category*)malloc(sizeof(Category));
    if (new_node == NULL) return NULL;

    printf("Enter Category ID: ");
    scanf("%d", &new_node->id);
    printf("Enter Category Code: ");
    scanf("%s", new_node->code);

    new_node->next = NULL;
    return new_node;
}

void insert_category(Category **tete) {
    Category* new_cat = create_category();
    if (new_cat) {
        new_cat->next = *tete;
        *tete = new_cat;
    }
}
```

3.2 Update_category

```c
void update_category(Category *tete, int id) {
    Category *current = tete;
    while (current != NULL) {
        if (current->id == id) {
            printf("Enter New Code: ");
            scanf("%s", current->code);
            return;
        }
        current = current->next;
    }
}
```

3.3 Delete_category

```c
void delete_category(Category **tete, int id) {
    Category *temp = *tete, *prev = NULL;
    if (temp != NULL && temp->id == id) {
        *tete = temp->next;
        free(temp);
        return;
    }
    while (temp != NULL && temp->id != id) {
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL) return;
    prev->next = temp->next;
    free(temp);
}
```

4. Linked List Explanation

Categories are stored using a linked list:

- Each node represents a category
- Nodes are connected using pointers
- Allows dynamic insertion and deletion

5. Repository

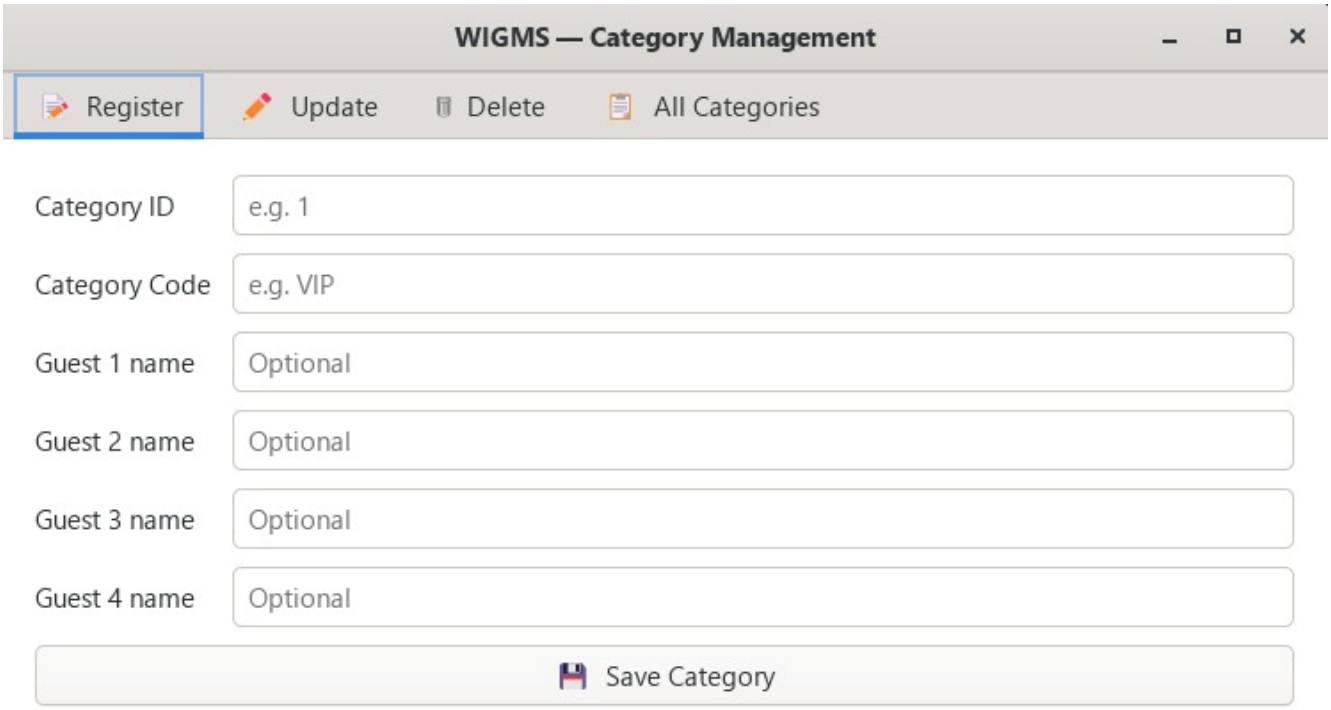
The GitHub repository was created and validated by all group members. The project can be accessed using the following link: https://github.com/C-programming-project-1/C-PROGRAMMING-PROJECT-REPOSITORY

6. Graphic User Interface (GUI's)

6.1 Location of the file

Name	Date modified	Type	Size
WIGMS	3/26/2026 8:28 AM	File folder	
.bash_history	3/26/2026 8:56 AM	BASH_HISTORY File	1 KB
.bash_profile	3/6/2026 4:25 PM	Fichier source Bas...	2 KB
.bashrc	3/6/2026 4:25 PM	Fichier source Bas...	6 KB
.profile	3/6/2026 4:25 PM	Fichier source Prof...	1 KB
csv_manager.c	3/25/2026 6:40 AM	C Source File	1 KB
marriage.c	3/26/2026 12:36 PM	C Source File	3 KB
person.c	3/25/2026 6:28 AM	C Source File	15 KB
person.exe	3/25/2026 6:12 AM	Application	164 KB
persons.csv	3/25/2026 6:42 AM	Fichier source Co...	1 KB

6.2 Presentation of the Category user interface



7. Tools and Technologies Used

- **C Programming Language for software development.**

- **GitHub platform for version control and team collaboration.**

- **Standard C libraries such as stdio.h and string.h**

8. Difficulties Encountered and Proposed Solutions

During the development of the Wedding Invitation and Gift Management System, the group encountered several general challenges that affected both Module 1 and Module 2. These challenges were related to programming concepts, system design, and teamwork. However, appropriate solutions were applied to ensure the successful progress of the project.

8.1 Difficulty in Managing Dynamic Data Structures

The implementation of linked lists in Module 2 introduced another challenge. Unlike static data structures such as arrays, linked lists require proper handling of memory allocation and pointer connections between nodes.Errors such as incorrect linking of nodes, loss of data, and program crashes occurred during testing.

Proposed Solution:

The group used diagrams to visualize how linked lists work and how nodes are connected. Step-by-step tracing of the code was performed to identify errors. Additional practice on linked lists helped improve understanding and correct implementation.

8.2 Difficulty in Handling User Input

Managing user input using functions like scanf () was problematic, especially when handling both strings and integers. Issues such as skipped inputs and incorrect data reading were encountered.

Proposed Solution:

The group reviewed proper input handling techniques and improved the structure of input statements. Additional validation checks were introduced to ensure correct data entry. Extensive testing was conducted to eliminate input errors.

8.3 Difficulty in Time Management and Task Coordination

Coordinating tasks among group members and meeting deadlines was sometimes difficult due to different schedules and workloads.

Proposed Solution:

Tasks were clearly assigned to each member, and deadlines were set for each phase of the project. Regular communication and progress tracking ensured that the work was completed on time.

9. Next Steps of the Project

The next phase will focus on:

- Implementing Priority Management (sorting algorithms)
- Developing Gift Management module
- Designing the User Interface
- Testing and debugging of system.

10. Conclusion

This second report highlights both the correction of Module 1 and the successful implementation of Module 2.

The project has significantly improved the group's understanding of:

- Structures
- Pointers
- Linked lists
- Modular programming

Additionally, the introduction of a user-friendly interface has made the system more interactive and efficient.

This module lays a strong foundation for the continuation of the project.